Convolutional Neural Network-Aided Tree-Based Bit-Flipping Framework for Polar Decoder Using Imitation Learning

Chieh-Fang Teng, *Student Member, IEEE*, and An-Yeu (Andy) Wu, *Fellow, IEEE*

***Abstract*—Known for their capacity-achieving abilities and low complexity for both encoding and decoding, polar codes have been selected as the control channel coding scheme for 5G communications. To satisfy the needs of high throughput and low latency, belief propagation (BP) is chosen as the decoding algorithm. However, it suffers from worse error performance than that of cyclic redundancy check (CRC)-aided successive cancellation list (CA-SCL). Recently, convolutional neural network-aided bit-flipping (CNN-BF) is applied to BP decoding, which can accurately identify the erroneous bits to achieve a better error rate and lower decoding latency than prior critical-set bit-flipping (CS-BF) mechanism. However, successive BF, having better error correction capability, has not been explored in CNN-BF since the more complicated flipping strategy is out of the scope of supervised learning. In this work, by using imitation learning, a convolutional neural network-aided tree-based multiple-bits BF (CNN-Tree-MBF) mechanism is proposed to explore the benefits of multiple-bits BF. With the CRC information as additional input data, the proposed CNN-BF model can further reduce 5 flipping attempts. Besides, a tree-based flipping strategy is proposed to avoid useless flipping attempts caused by wrongly flipped bits. From the simulation results, our approach can outperform CS-BF and reduce flipping attempts by 89% when code length is 64, code rate is 0.5 and SNR is 1 dB. It also achieves a comparable block error rate (BLER) as CA-SCL.**

***Index Terms*—Polar codes, belief propagation, bit-flipping, convolutional neural network, imitation learning.**

I. Introduction

POLAR code is a type of block channel code proven to achieve channel capacity [1]. In recent years, it has received intensive attention due to its low complexity for encoding and decoding. In 2016, it was selected by 3GPP as the officially coding scheme for the enhanced mobile broadband (eMBB) control channel of 5G New Radio (NR) [2].

The two main algorithms for polar decoding are successive cancellation (SC) and belief propagation (BP). Compared with BP decoding, SC decoding can fulfill the channel-capacity ability and achieve a lower block error rate (BLER) through enhanced SC algorithms, such as SC list (SCL) [3]–[4] and SC flip (SCF) [5]–[13]. However, SC suffers from high latency and low throughput due to its inherently sequential processing nature, while BP algorithm excels in architectural parallelization, thus has lower decoding latency and better throughput [14]. Recently, considerable efforts have been put into improving the error performance of BP to achieve that of enhanced SC algorithms, while still maintaining its advantages.

In [15]–[17], the BP algorithm is enhanced through the scaling of messages from trainable weights. It can reduce the total number of BP iterations with lower overall complexity, but still does not address the lacking error correction performance. The authors in [18] concatenate the CRC factor graph with the polar factor graph for the exchanging of extrinsic information, which can achieve better error correction performance. A BP list (BPL) decoder is proposed in [19]–[21] to achieve comparable performance as SCL by performing BP algorithm on different permuted factor graphs or permuted codewords. In [22], post-processing is applied to different types of BP errors by performing enhancement and perturbation on the reliable and unstable bits, respectively. A similar concept was proposed in [23] by iteratively finding the stable bits and strengthening them as "frozen bits". However, even with these enhanced approaches, all of them still can not achieve comparable performance as cyclic redundancy check (CRC)-aided SCL (CA-SCL) [3]–[4].

Recently, bit-flipping (BF) decoders also attract a lot of attention because it can minimize the effect of error propagation by performing error corrections on incorrectly decoded bits during decoding iterations [5]–[13], [24]–[29]. In [5]–[13], BF has been successfully applied to SC, but it further deteriorates the decoding latency. On the other hand, with the aid of BF, the BLER of BP-based decoders can be comparable with that of CA-SCL at low to medium SNRs [24]–[25]. Moreover, BP flip (BPF) decoders are realized in [27]–[28] to demonstrate its great throughput and comparable performance as SCL. Recently, as the fast-emerging field of deep learning-assisted communication systems, many researchers start exploiting the benefit of deep learning for the design of channel coding [30]–[32]. A convolutional neural network-aided BF (CNN-BF) was proposed in [29], which can dynamically identify the erroneous bits from the metadata of BP decoding as shown in Fig. 1(a). Thus, it can achieve better error correction capability and lower decoding

Manuscript received April 16, 2020; revised August 28, 2020 and October 19, 2020; accepted November 24, 2020. Date of publication November 27, 2020; date of current version January 5, 2021. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Chuan Zhang. This work was supported by MediaTek, Inc., Hsinchu, Taiwan, under Grant MTKC-2019-0070. The work of Chieh-Fang Teng was sponsored by MediaTek Ph.D. Fellowship program. *(Corresponding author: An-Yeu (Andy) Wu.)*

The authors are with the Graduate Institute of Electronics Engineering and Department of Electrical Engineering, National Taiwan University, Taipei 10617, Taiwan (e-mail: jeff@access.ee.ntu.edu.tw; andywu@ntu.edu.tw).

Digital Object Identifier 10.1109/TSP.2020.3040897

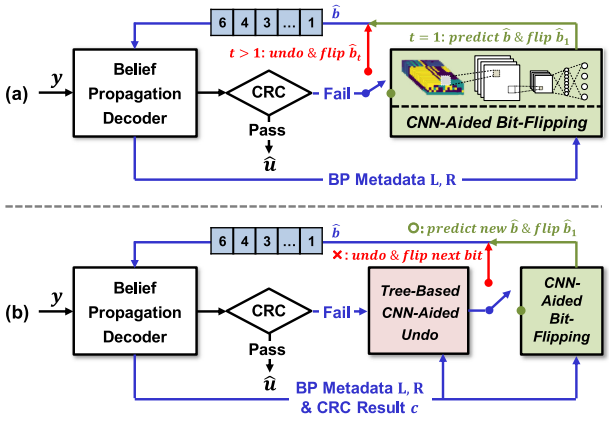

Fig. 1. Convolutional neural network-aided bit-flipping for polar decoder: (a) 1-bit bit-flipping [29], and (b) proposed multiple-bits bit-flipping.

latency than [24]. However, some worse codewords that demand successive flipping attempts cannot be successfully decoded in [29]. To further explore the benefit of BF and the great potential of deep learning, we extend [29] from 1-bit BF to multiple-bits BF as shown in Fig. 1(b). However, two critical issues should be addressed:

1) *Early termination mechanism*: For the BF mechanism, it sets the a priori knowledge of information bit to infinity to avoid error propagation. However, for multiple-bits BF, suppose the last flipped bit is incorrectly set as a frozen bit, the following flipping attempts are useless. Thus, it requires an effective mechanism to determine whether to continuously flip the next bit or undo the wrongly flipped bit.
2) *Deficiency of supervised learning*: For 1-bit BF strategy, if the flipped bit cannot result in correct decoding, it always undoes the action and sequentially attempts other candidates. Thus, the subsequent flipping is independent with the previous attempts and supervised learning can be easily applied for the training. However, as the mechanism extended to multiple-bits BF, the next flipping action is based on the current decoding state and flipping action, which means that the training data cannot be easily obtained and is out of the scope of supervised learning.

In this paper, by taking advantage of imitation learning, we propose a convolutional neural network-aided tree-based multiple-bits BF (CNN-Tree-MBF) mechanism, which can significantly improve the error correction capability and reduce decoding latency by exploring the benefit of multiple-bits BF. Our main contributions are summarized as below:

1) *Design of convolutional neural network-aided bit-flipping model*: To further enhance the prediction accuracy of the CNN-BF model, we transform the CRC results to an image as the additional input data for the CNN model. By doing so, it can further reduce 5 flipping attempts compared to the CNN-BF model used in [29].
2) *Design of tree-based undo model*: A tree-based Undo model for flipping strategy is proposed to strike a good balance between the benefit of multiple-bits BF and the increased flipping attempts caused by wrongly flipped bits. Besides, a CNN-aided Undo model is also proposed, which can be combined into the tree-based Undo model to further reduce flipping attempts by 19%.
3) *Multiple-bits bit-flipping mechanism using imitation learning*: A multiple-bits BF mechanism, composed of the aforementioned BF model and the Undo model, is proposed as shown in Fig. 1(b). By iteratively alternating between the stages of training and data generation, these two models can be well-trained by imitation learning. From the simulation results, it can outperform state-of-the-art critical set bit-flipping (CS-BF) algorithm [24] with 89% reduction in flipping attempts. Besides, by increasing the maximum number of flipping attempts, it can achieve comparable performance as CA-SCL with slightly increased average decoding latency.

The rest of this paper is organized as follows. Section II briefly reviews BP decoding and prior BF works. In Section III, after detailed analyses of the benefits and challenges of multiple-bits BF, we propose our multiple-bits BF mechanism. Based on this mechanism, the design of the CNN-BF model and the tree-based Undo model are shown in Section IV. The numerical experiments and complete analyses are shown in Section V. Finally, Section VI concludes our work.

II. Polar Codes and Decoding Algorithms

A. Polar Codes

To construct an $(N,\ K)$ polar code, the N-bits message $\boldsymbol{u}$ is recursively constructed from a 2×2 polarizing transformation $\mathbf{F} = \left[\begin{smallmatrix} 1 & 0 \\ 1 & 1 \end{smallmatrix}\right]$ by $\log_2 N$ times to exploit the channel polarization [1]. As $N \to \infty$, the synthesized channels tend to two extremes: the noisy channels (unreliable) and noiseless channels (reliable). Therefore, the K information bits are first assigned to the K most reliable bits in $\boldsymbol{u}$ and the remaining $(N - K)$ bits are referred to as frozen bits with the assignment of zeros. Besides, the r-bits cyclic redundancy check (CRC) is attached as the information bits, which can be utilized to check the correctness of decoded results. Thus, the actual information rate is $(K - r)/N$. Then, the N-bits transmitted codeword x can be generated by multiplying $\boldsymbol{u}$ with generator matrix $\mathbf{G}$ as follow:

$$\boldsymbol{x} = \mathbf{G}\boldsymbol{u} \ = \mathbf{F}^{\otimes n}\,\mathbf{B}\boldsymbol{u},\ n \ = \log_2 N\ . \tag{1}$$

$\mathbf{F}^{\otimes n}$ is the n-th Kronecker power of $\mathbf{F}$ and $\mathbf{B}$ represents the bit-reversal permutation matrix.

B. Belief Propagation Decoding Algorithm

Belief propagation is a widely used message-passing algorithm for decoding, such as in low-density parity-check (LDPC) codes and polar codes. The decoding process of polar codes is to iteratively apply BP algorithm over the corresponding factor graph as shown in Fig. 2. For an (N, K) polar code, there are $n \ = \log_2 N$ stages and a total of $N \times (n + 1)$ nodes on the factor graph. Each node $(i,\ j)$ represents the j-th node at the i-th stage in the factor graph. It has two types of log likelihood ratios (LLRs), namely left-to-right message $\mathbf{R}_{i,j}^{(t)}$ and

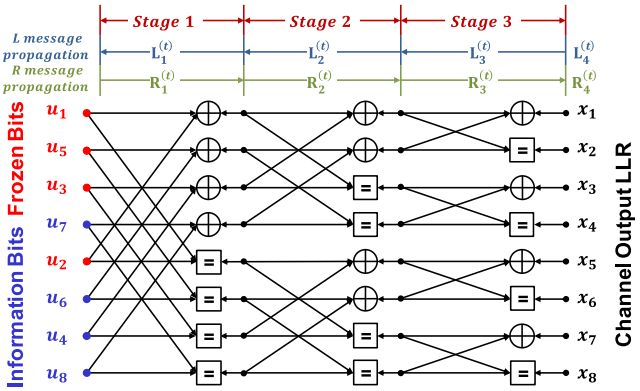

Fig. 2. Factor graph of polar codes with $K = 4$ and $N = 8$. $A = \{4, 6, 7, 8\}$ and $A^c = \{1, 2, 3, 5\}$.

right-to-left message $\mathbf{L}_{i,j}^{(t)}$, where t represents the t-th iteration. Before beginning the iterative propagation and the updating of node values, their LLR values are first initialized as:

$$\mathbf{R}_{1,j}^{(1)} = \begin{cases} 0, & \text{if } j \in A \\ +\infty, & \text{if } j \in A^c \end{cases}, \quad \mathbf{L}_{n+1,j}^{(1)} = \ln \frac{P\left(\boldsymbol{y}_j | \boldsymbol{x}_j = 0\right)}{P\left(\boldsymbol{y}_j | \boldsymbol{x}_j = 1\right)}, \tag{2}$$

where A and A^c are the set of information bits (including CRC bits) and the set of frozen bits, respectively.

Then, the iterative decoding procedure with the updating of $\mathbf{R}_{i,j}^{(t)}$ and $\mathbf{L}_{i,j}^{(t)}$ is given by:

$$\begin{cases} \mathbf{L}_{i,j}^{(t)} = g\left(\mathbf{L}_{i+1,j}^{(t)}, \mathbf{L}_{i+1,j+N/2^i}^{(t-1)} + \mathbf{R}_{i,j+N/2^i}^{(t)}\right), \\ \mathbf{L}_{i,j+N/2^i}^{(t)} = g\left(\mathbf{R}_{i,j}^{(t)}, \mathbf{L}_{i+1,j}^{(t)}\right) + \mathbf{L}_{i+1,j+N/2^i}^{(t)}, \\ \mathbf{R}_{i+1,j}^{(t)} = g\left(\mathbf{R}_{i,j}^{(t)}, \mathbf{L}_{i+1,j+N/2^i}^{(t-1)} + \mathbf{R}_{i,j+N/2^i}^{(t)}\right), \\ \mathbf{R}_{i+1,j+N/2^i}^{(t)} = g\left(\mathbf{R}_{i,j}^{(t)}, \mathbf{L}_{i+1,j}^{(t-1)}\right) + \mathbf{R}_{i,j+N/2^i}^{(t)}, \end{cases} \tag{3}$$

where $g(a,\ b) \approx \text{sign}(a)\text{sign}(b)\min(|a|,\ |b|)$ is the min-sum approximation introduced to reduce complexity. Finally, after T iterations, the estimation of $\hat{\boldsymbol{u}}$ is decided by:

$$\hat{\boldsymbol{u}}_j = \begin{cases} 0, & \text{if } \mathbf{L}_{1,j}^{(T)} + \mathbf{R}_{1,j}^{(T)} \geq 0, \\ 1, & \text{if } \mathbf{L}_{1,j}^{(T)} + \mathbf{R}_{1,j}^{(T)} < 0. \end{cases} \tag{4}$$

With the CRC, the decoded results can be checked by multiplying with the CRC parity check matrix $\mathbf{H}_{CRC}$ as follows:

$$\boldsymbol{c} = \mathbf{H}_{CRC}\, \hat{\boldsymbol{u}}. \tag{5}$$

If $\boldsymbol{c}$ is a zero vector, it means that the decoded results pass CRC; otherwise, the codeword fails to decode successfully.

C. Prior Work: Belief Propagation Bit-Flipping Decoding Algorithms [24]–[29]

Due to the message passing algorithm of BP decoding, the incorrectly estimated information bits may result in error propagation and thus negatively affect the reliability of many other bits. To address the issue of error propagation, BF is an assistive mechanism to the decoding process, where a possibly incorrectly decoded bit is guessed and flipped prior to the restarted decoding process. Thus, precise BF can effectively improve the block error rate (BLER) performance for polar codes. The mechanism of BF flips the value of previous estimated $\hat{\boldsymbol{u}}_j$, and sets the a priori knowledge of $\boldsymbol{u}_j$ to infinity as if it is a frozen bit. Therefore, the initialized values of $\mathbf{R}_0$ in (2) are revised as:

$$\mathbf{R}_{1,j}^{(1)} = \begin{cases} 0, & \text{if } j \in \{A \backslash F\} \\ \infty \times \left(2\hat{\boldsymbol{u}}_j - 1\right) \times \boldsymbol{d}_j, & \text{if } j \in F \\ +\infty, & \text{if } j \in A^c \end{cases}, \tag{6}$$

where F is the set of current flipped bits and thus the bits in F will be set to infinity. $\boldsymbol{d}_j \in \{-1,\ 1\}$ represents the flipping direction. If $\boldsymbol{d}_j = 1$, it means to flip the bit $\hat{\boldsymbol{u}}_j$ to the opposite direction of $\mathbf{L}_{1,j}^{(T)} + \mathbf{R}_{1,j}^{(T)}$. Vice versa, it strengthens the bit $\hat{\boldsymbol{u}}_j$ to the same direction. By doing so, the a priori knowledge of flipped bits is expected to correct the other wrongly propagated messages in the previously failed BP decoding.

According to the algorithm detailed above, the decoding latency of BF corresponds to the required number of flipping attempts, which is dominated by the correction of flipped bits. Therefore, critical set (CS), consisting most of the error-prone bits, was proposed and adopted in [7]–[8], [10]–[11], [24]–[25]. The critical set is constructed based on the structure of polar code, where the first nodes in the subtrees of all information bits are at high risk, and thus are included in the set. By only selecting bits for BF from CS, it results in less flipping attempts and achieves lower latency. Furthermore, CS with order ω is proposed in [24] to flip ω error-prone bits simultaneously, which has better error correction capability at the cost of 2^ω times increase in flipping attempts. In [25], error types for BF are analyzed and multiple bit-flipping sets are dynamically generated based on the submatrix check, which can remove unnecessary flipping positions and increase the order of CS.

Though critical set can effectively reduce the search space for flipping attempt, this mechanism is essentially still a process of trial-and-error to attempt all the bits in critical set. Besides, it also excludes error bits outside the critical set, thus degrading the error correction capability of BF. In [27], BP flip decoder is realized with efficient identification of error-prone bits, which is based on the magnitude of LLR values and 3GPP standard. Furthermore, [27] is extended in [28] by presenting a hardware-friendly high-order BF mechanism. Recently, a convolutional neural network-aided bit-flipping (CNN-BF) was proposed in [29] to exploit the variation of BP decoding process and dynamically identifies the erroneous bit for 1-bit correctable codewords, which can achieve better decoding performance and lower flipping attempts compared to [24].

After the flipping order is established from CS [24] or predicted by CNN model [29], the conventional BP decoding process can commence. If BP fails to decode successfully, checked by a cyclic redundancy check (CRC), a candidate bit from the flipping order is sequentially selected for flipping according to Eq. (6) as shown in Fig. 1(a). After BF, BP decoding is performed again. If the result satisfies CRC, the decoding process is completed. Otherwise, it undoes the last flipped bit and attempts the other candidates in the flipping order. The process is iteratively performed until CRC is successfully passed. For more details about BF and critical set, please refer to [5]–[13], [24]–[29].

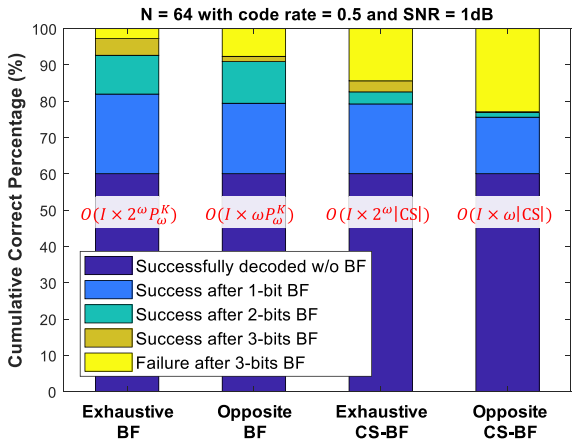

Fig. 3. Analysis of error correction capability and time complexity between different bit-flipping mechanisms.

III. Proposed Multiple-Bits Bit-Flipping Mechanism Using Imitation Learning

A. Analysis of Different Bit-Flipping Mechanisms

Before introducing our proposed multiple-bits BF, we first evaluate the error correction capability and time complexity between four different BF mechanisms as shown in Fig. 3. In Fig. 3, the analysis of CS-BF is based on [24], which means that only the bits in CS are considered for flipping. Therefore, the time complexity is proportional to the size of the critical set |CS|. Besides, ω denotes the number of simultaneously flipped bits. On the other hand, without constrained by CS, it can flip arbitrary bits in the set of information bits. Furthermore, another difference is that it adopts a progressive strategy, which means it progressively increases the number of flipped bits to ω by flipping one bit at a time. For example, suppose $\omega = 2$, it will firstly flip one bit followed by BP decoding. If it still fails to decode successfully, it continuously flips another bit and applies BP decoding again. This strategy is more consistent with our scenario to apply neural networks to predict the error probability for each bit and only flip the bit with the highest error probability. Therefore, the time complexity for this progressive strategy is proportional to P_{ω}^{K} instead of C_{ω}^{K}, where P and C represent permutation and combination, respectively.

The term of "exhaustive" means to flip the bits in both directions, namely with $\boldsymbol{d}_j \in \{-1, 1\}$. Thus, it has 2^{ω} different combinations for flipping directions and the time complexity is proportional to 2^{ω}. Vice versa, the term of "opposite" means that it only attempts the opposite direction with $\boldsymbol{d}_j = 1$, which can effectively reduce the flipping attempts at the cost of degraded error correction capability. However, its time complexity is reduced to only proportional to ω. In Fig. 3, the maximum number of flipped bits ω is set to 3 and I denotes the number of iterations for each BP decoding. Because each BF strategy will attempt all flipping combinations until it correctly decodes the codeword, we can obtain the best performance that each strategy can achieve. Besides, the error correction capability of smaller ω is included in bigger ω. Thus, we use the metric of cumulative correct percentage to evaluate the improved error correction capability as the increase of ω.

In Fig. 3, the performance of exhaustive BF can be seen as the optimal performance due to the greatest searching space includes all combinations. However, the performance of opposite BF is only slightly worse than that of exhaustive BF, which means that flipping the bits to the opposite direction is a more efficient strategy to achieve good performance at a reasonable number of flipping attempts. On the other hand, for the performance of CS-based mechanisms, we can observe that the improvement by increasing ω is limited, especially for opposite CS-BF. This is due to the constrained searching space and it is hard to simultaneously flip multiple bits to the correct directions without exhaustive searching.

In [29], the authors adopted the mechanism of opposite 1-bit BF and dedicated to reducing the time complexity from $O(I \times P_1^K)$ to $O(I)$ by improving the accuracy of the flipping order. However, from Fig. 3, we can observe that there are about 20% error codewords can not be corrected by opposite 1-bit BF. Therefore, we want to extend [29] from 1-bit BF to multiple-bits BF in this work. By doing so, the error correction capability of BF can be greatly improved and the superiority over CS-BF will also be more apparent.

B. Challenges for Neural Network-Aided Multiple-Bits Bit-Flipping

For the training of the neural network-aided 1-bit BF model, it can be easily divided into two steps. Firstly, collecting all the received codewords, which fail to decode successfully after the first BP decoding. Secondly, we need to label the correct flipping position for these codewords. The process exhaustively goes through the K information bits by setting the a priori knowledge of $\boldsymbol{u}_j$ to opposite infinity as a frozen bit and followed by BP decoding. If the newly decoded result is correct, the label $\boldsymbol{b}_j$ is set to 1; vice versa, it is labeled to 0. In this way, the labeled data for these failed decoded codewords can be obtained and the neural network can be trained using supervised learning.

However, the above flow is only applicable to 1-bit BF model. Because if the decoded result still fails to pass CRC, it will undo the last flipped bit and attempt other candidates in the flipping order. Thus, the subsequent flipping is independent with the previous attempts. However, as the mechanism extended to multiple-bits BF, the next flipping action is based on the current decoding state and current flipping action, which is out of the scope of supervised learning. For example, the next flipped position can be classified into three categories:

- **Correct position**: after flipping this position, the next BP decoding will successfully decode the codeword, which is labeled to 1.
- **Wrong position**: the flipped position will cause unrecoverable error bit (e.g.,: $\boldsymbol{u}_j$ is 1, but $\mathbf{R}_{1,j}^{(1)}$ is set to $+\infty$), which is labeled to 0.
- **Intermediate position**: the flipped position will not cause unrecoverable error bit (e.g.,: $\boldsymbol{u}_j$ is 0 and $\mathbf{R}_{1,j}^{(1)}$ is set to $+\infty$), but the next BP decoding still fails to decode the codeword successfully, which is also labeled to 0.

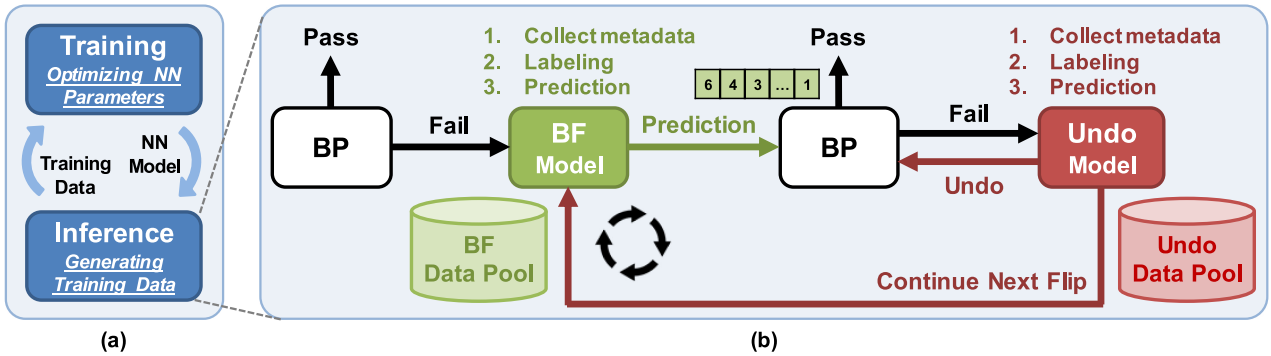

Fig. 4. The overview of the proposed multiple-bits bit-flipping mechanism using imitation learning: (a) the iterative process between training and inference stages; (b) the detailed flow chart of the proposed multiple-bits bit-flipping mechanism at the inference (training data generation) stage.

Therefore, the next action is based on the current state and current action. Suppose the current action flips the wrong position, the next action must undo to escape from the unrecoverable state. Besides, the different intermediate positions will also result in different correct positions for the next action. This challenge was also firstly pointed out in [12], where the authors adopted a long short-term memory (LSTM) network to continuously predict the first error bit for SC decoding. Similarly, the previous action will also affect the following action in that scheme.

In conclusion, multiple-bits BF will correlate the previous actions and the subsequent actions and thus is out of the scope of supervised learning. The problem of multiple-bits BF is just like a multi-step maze game or a self-driving car problem, where multiple decisions are made for a success to the destination. The mission of our designed neural network model is to find out the correct action (flipping position) based on the state (designed input data) in each step. Consequently, to address this challenge, we need to exploit imitation learning, which has been successfully applied to the training of self-driving cars [33].

C. Multiple-Bits Bit-Flipping Using Imitation Learning

According to the previous discussions, we need two kinds of models, namely the *BF model* and the *Undo model*, to accomplish the multiple-bits BF mechanism. The BF model is used to predict the error probability for each bit and the Undo model is utilized to indicate the correctness of last BF action and whether an undo action is required to rescue the wrongly flipped bit. Therefore, it is difficult to train these two models using supervised learning due to the action of these two models will serve as the input for each other and thus affect their actions in the next time step as shown in Fig. 4(b).

To address this issue, imitation learning is adopted to iteratively generate the training data and optimize the models' parameters as shown in Fig. 4. To be precise, our approach is direct policy learning (DPL) in imitation learning, which directly learns from expert demonstrations. In the stage of training data generation, we need to simulate a real inference environment for these two models to interact with each other to avoid data mismatch between training and inference. The flow chart of the multiple-bits BF mechanism can be mainly divided into three states as shown in Fig. 4(b) and described below:

1) Bit-flipping prediction: Firstly, the received codewords will be decoded by BP algorithm. Then, the failed decoded codewords will be sent to the BF model. In this state, if generating training data is required, the BP's metadata and label $\boldsymbol{b}$, which can be obtained according to Section III.B, will be jointly collected into the BF data pool as the training data pair. Then, the BF model predicts the error probability for each bit to construct the flipping order.

2) Belief propagation decoding: Next, the BP decoding will be executed again after flipping the bit with the highest error probability. Then, the failed decoded codewords will enter into the Undo model.

3) Undo prediction: In this state, if generating training data is enabled, the process of labeling and collecting BP's metadata will be performed. However, the labeling process is slightly different from the process for the BF model. In this part, if the last flipped bit belongs to the wrong position, the label m is set to 1; vice versa, the position must belong to the intermediate position and thus be labeled to 0. Then, the label m and BP's metadata are also collected into the Undo data pool. Finally, the Undo model predicts the error probability of the last flipped position. If the probability is greater than 0.5, it will undo the last flipped bit and attempt the next bit in the flipping order. On the other hand, it will enter the BF model again for the training data collection and prediction based on the updated metadata from the newly performed BP decoding.

The above process will be iterated until enough training data is generated or the stopping criteria for multiple-bits BF mechanism are met. After collecting enough training data, the stage will transit from inference to the training stage as shown in Fig. 4(a). The BF model and the Undo model will be optimized based on the training data in the data pool. Because the model's behavior will change after training, the training data in the data pool must also be updated. Thus, the process will transit to the inference stage again. By iteratively using imitation learning for the process of training and data generation, these two models can converge and learn to accomplish the multiple-bits BF mechanism. This iterative process will continue until the accuracy on the validation set cannot be further improved, which means that both models have converged to a steady point. The detailed algorithm flow will be provided in the next section.

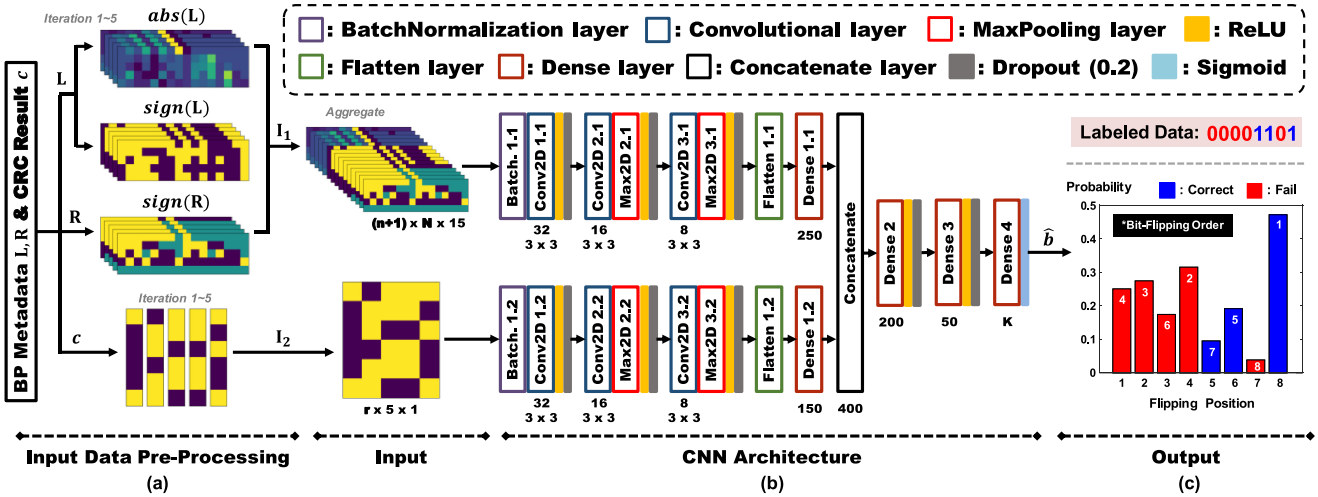

Fig. 5. The detailed overview of the proposed convolutional neural network-aided bit-flipping model with $(N, K, r) = (16, 8, 6)$: (a) illustration of the input data and data pre-processing; (b) proposed convolutional neural network architecture; (c) illustration of labeled data and bit-flipping order based on model's prediction results.

IV. Proposed Bit-Flipping Model and Undo Model Designs

After completing the design of the multiple-bits BF mechanism shown in Fig. 4(b), the remaining question is how to design the BF model and the Undo model, which has a great impact on the error correction capability and the required number of flipping attempts. The better prediction accuracy of the BF model can accurately identify the erroneous bits and thus improve the error correction capability and reduce the flipping attempts. On the other hand, the Undo model can do early termination to avoid useless flipping attempts and rescue the codewords from an unrecoverable state. In the following, we will describe the design of these two models in detail.

A. Convolutional Neural Network-Aided Bit-Flipping Model

Firstly, we focus on the design of the BF model. The input data for the model-based approach is important since it has a significant impact on prediction accuracy. Compared with [29], we not only make use of the metadata from BP, but further take advantage of the CRC results $\boldsymbol{c}$ as the input data for the training and prediction.

In each BP decoding iteration, the values of LLRs $\mathbf{L}^{(t)}$, $\mathbf{R}^{(t)}$ on the whole factor graph are recorded and mapped to an image as shown in Fig. 5(a). Besides, the decoded results $\hat{\boldsymbol{u}}^{(t)}$ and CRC results $\boldsymbol{c}^{(t)}$ for each BP iteration can be obtained according to Eq. (4) and Eq. (5), respectively. The values $\mathbf{L}^{(t)}$, $\mathbf{R}^{(t)}$, and $\boldsymbol{c}^{(t)}$ are preprocessed and jointly used as the input data for the following neural network model. Though CRC results $\boldsymbol{c}^{(t)}$ can only be utilized for error-detection, it may potentially indicate which information bits are possibly erroneous because each bit in $\boldsymbol{c}^{(t)}$ is connected with part of the information bits. Suppose $\boldsymbol{c}_i^{(t)}$ is 1, it means that at least one of its connected information bits is wrongly decoded. Thus, it is informative for neural networks to further analyze and dynamically identify the most likely error bits with greatly improved prediction accuracy, which will be demonstrated in Section V. Also, due to the iterative decoding process, the images and CRC results, representing different iterations, will be jointly integrated as the input data. Therefore, the adopted model can explore not only the relation between connected nodes but also the variation of LLRs and CRC results among different iterations, namely in both spatial and temporal dimensions, which further enhance the prediction accuracy.

In addition, to further improve the prediction accuracy and reduce model complexity, we apply some domain-specific signal pre-processing for the LLR values before feeding the input data into the model. Two features, the absolute and sign values, are extracted from LLRs as shown in Fig. 5(a). By doing so, the absolute values represent the reliability of each node and the sign values are helpful for the model to further explore the variation between different nodes. Besides, since the values of $abs(\mathbf{R})$ are around either 0 or ∞, which are not suitable for the concatenated model to extract features. Thus, the images of $abs(\mathbf{R})$ are removed without affecting the prediction accuracy but reducing the model complexity. Suppose that the number of iterations for BP is 5, there will be a total 15 images after data pre-processing with each image resolution being $(n+1) \times N$, which is consistent with the size of the factor graph. The CRC results in each BP iteration are also integrated as an image with the resolution being $r \times 5$ as shown in Fig. 5(a), which is very suitable for neural networks to analyze the variation in the temporal domain and figure out error-prone bits.

Two convolutional neural networks (CNNs) are employed to deal with the image-based input data $\mathbf{I}_1$ and $\mathbf{I}_2$, which correspond to the preprocessed factor graphs and CRC results, respectively. CNN is the widely used model for image processing with the ability to extract local connectivity and subtle features of the input images. After feature extraction, these two CNNs will be concatenated to jointly predict the error probability as shown in Fig. 5(b). Because our problem is not as complicated as in computer vision, it is unnecessary to construct a massive CNN model, such as AlexNet or ResNet. Thus, two tiny CNN models with similar architecture are proposed as shown in Fig. 5(b). For each CNN model, it is mainly constructed by three two-dimensional

Algorithm 1: Proposed CNN-Aided Tree-Based Multiple-Bits Bit-Flipping (CNN-Tree-MBF) Decoder.

Input: $\boldsymbol{y}, A, d, \boldsymbol{w}, T_{max}, \mathbf{H}_{CRC}, (\boldsymbol{u})$
Output: $\hat{\boldsymbol{u}}$

```
L, R ← initialize the BP decoder using (2)
û, L, R ← BP decoder(L, R)
t ← 0
l ← 1
if û does not pass CRC and t < T_max then
    û_old ← û
    Multiple-Bits BF Subtree(L, R, û_old, t, l, (u))
else
    return û
end if
```

convolutional layers for high-level feature extraction and one dense layer for feature transformation. After concatenating the extracted features from these two CNN models, three dense layers are followed for classification based on the nonlinear combinations of these extracted features from CNN models.

The values below the convolutional layer represent the number and size of filters, respectively. On the other hand, the values below the dense layer represent the number of nodes. Because of the image resolution of $\mathbf{I}_2$ is smaller than $\mathbf{I}_1$, we set a smaller number of nodes for the dense layer of $\mathbf{I}_2$ to reduce the number of parameters. Besides, a batch normalization layer is applied to the input images to normalize the input data and improve the convergence speed. Two maxpooling layers are also used to reduce the dimensionality of the feature map for the reduction of computational complexity and memory overhead. The nonlinear activation function, Rectified Linear Units (ReLUs), among each layer is defined as:

$$f_{ReLU}\ (x) = \max\{0, x\}\ . \tag{7}$$

It is helpful for extracting more complex features. Besides, to reduce overfitting, the regularization technique of "dropout" that avoids updating the weights of part nodes, is also utilized to improve the inference accuracy.

For the problem of BF prediction, the output layer has K nodes, which represents the probability of each bit being flipped or not. The labeled data for training is a vector with K values being 0 or 1 to indicate which bits could be flipped to result in successful decoding as shown in Fig. 5(c). Note that for some input cases, there could be more than 1 position to result in successful decoding. Consequently, this is a multi-label classification problem and the output must be rescaled into the range [0, 1] with sigmoid function to indicate the probability as below:

$$f_{Sigmoid}(x) = \sigma(x) = \left(1 + e^{-x}\right)^{-1}\ . \tag{8}$$

Also, the loss function is cross-entropy, defined as:

$$\mathcal{L}(\boldsymbol{b}, \hat{\boldsymbol{b}}) = -\frac{1}{K}\sum_{i=0}^{K-1} \boldsymbol{b}_i \log(\hat{\boldsymbol{b}}_i) + (1 - \boldsymbol{b}_i)\log(1 - \hat{\boldsymbol{b}}_i), \tag{9}$$

where $\boldsymbol{b}_i$ and $\hat{\boldsymbol{b}}_i$ denote the labeled data and predicted value for the i-th output, respectively.

B. Tree-Based Undo Model With Preorder Traversal

The Undo model is equally important as the BF model because the wrongly flipped bits can only be corrected by the Undo model. Suppose the Undo model can not effectively undo the wrongly flipped bits, the following flipping attempts are useless. On the other hand, suppose the last flipping attempt flips an intermediate position, the flipping process must be continued or it will degrade the correction capability. The previous flipping position and the action adopted by the Undo model can lead to four different cases:

- **True positive (TP)**: the last flipping attempt flips a wrong position and the Undo model correctly predicts "Undo". This will avoid useless flipping attempts.
- **True negative (TN)**: the last flipping attempt flips an intermediate position and the Undo model correctly predicts "No Undo". This will not degrade the error correction capability.
- **False positive (FP)**: the last flipping attempt flips an intermediate position, but the Undo model wrongly predicts "Undo". This will degrade the error correction capability.
- **False negative (FN)**: the last flipping attempt flips a wrong position, but the Undo model wrongly predicts "No Undo". The following flipping attempts will be useless.

Thus, the Undo model must strike a good balance to take advantage of the error correction capability from the multiple-bits BF mechanism while avoiding a significantly increased number of flipping attempts caused by wrongly flipped bits.

To satisfy the aforementioned requirements, we firstly propose a criteria-based Undo model, where the actions determined by the Undo model are based on a tree structure as shown in Fig. 6(a). The tree structure can be determined by two kinds of parameters. One is the maximum depth d, which represents the maximum number of successively flipped bits for each codeword and is equivalent to ω. Another one is the number of children $\boldsymbol{w}_i$ for the nodes in level i, representing $\boldsymbol{w}_i$ bits with the highest error probability will be attempted for each BF prediction. After determining the tree structure, the actions adopted by the Undo model are completely based on the tree's preorder traversal, which is a depth-first search to better explore the benefit of multiple-bits BF. Our approach has a similar concept as [6] and [28], but both of them are based on a breadth-first search. The "Undo" action is performed only when the multiple-bits BF mechanism traverses to the maximum depth; otherwise, it successively flips other bits based on a new BF prediction.

Take Fig. 6(a) as an example. The received codeword is first decoded by using BP algorithm. Suppose it fails to decode successfully, the BF model predicts the error probability as indicated by the red node and $\boldsymbol{w}_1$ bits with the highest error probability are kept in descending order. Based on the order, the position with index 6 is chosen for flipping and followed by BP decoding. Because it has not reached the maximum depth, the BF model predicts the error probability again based on the updated $\mathbf{L}$, $\mathbf{R}$, and $\boldsymbol{c}$. Then, the position with index 4 is flipped and followed by BP decoding. Now, because it has reached the maximum depth, the "Undo" action is performed. So, it returns to the red node and continuously attempts other $\boldsymbol{w}_2 - 1$ bits. Suppose it still fails, the "Undo" action is performed again and it returns to the level 1.

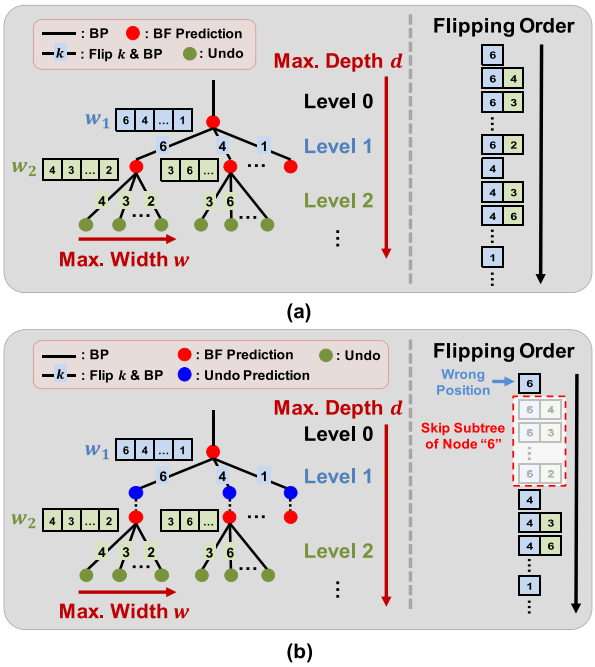

Fig. 6. The detailed overview of the tree-based Undo model and the corresponding flipping order with $d = 2$. (a) Tree-based Undo model; (b) Tree-based CNN-aided Undo model (suppose index 6 is wrong position).

Then, the position with index 4 is chosen and followed by a similar flow. Each row in the flipping order is corresponding to the set F in Eq. (6). The process is performed recursively until the codeword passes CRC or it completely traverses the entire tree or reaches the maximum traversal attempts T_{max}.

Based on the tree-based Undo model and the adjustment of the tree's maximum depth, we can exploit the benefit of multiple-bits BF without significantly increasing the number of flipping attempts. Besides, it is worth noting that [29] can be seen as a special case of this tree structure when the maximum depth d is set to 1.

C. Tree-Based Convolutional Neural Network-Aided Undo Model With Preorder Traversal

According to the above discussions, the proposed tree-based Undo model has a deficiency that the "Undo" action will be performed only when reaching the maximum depth. Suppose the wrongly flipped bit occurs in level 1, the subsequent flipping attempts on that subtree are all useless and thus result in longer decoding latency. To address this issue, we propose a tree-based CNN-aided Undo model based on the tree-based Undo model with a small modification as shown in Fig. 6(b). We can observe a blue node, representing a CNN-aided Undo model, is inserted before entering into the BF prediction in level 2. By taking advantage of this model, the wrongly flipped bit can be detected and early "Undo" action can be adopted to avoid the following useless flipping attempts. Suppose the position with index 6 belongs to the wrong position and the CNN-aided Undo model successfully identifies this bit, the following flipping attempts on the subtree can be skipped to reduce the decoding latency.

Algorithm 2: Multiple-Bits BF Subtree().

Input: $\mathbf{L}$, $\mathbf{R}$, $\hat{\boldsymbol{u}}_{old}$, t, l, $(\boldsymbol{u})$
Output: $\hat{\boldsymbol{u}}$

$\mathbf{I}_1, \mathbf{I}_2, \boldsymbol{c} \leftarrow$ input data pre-processing($\mathbf{L}$, $\mathbf{R}$, $\mathbf{H}_{CRC}$)
if *generate training data* **then**
  $\boldsymbol{b} \leftarrow$ label BF training data($\mathbf{L}$, $\mathbf{R}$, $\boldsymbol{u}$)
  BF data pool $\leftarrow$ ($\mathbf{I}_1$, $\mathbf{I}_2$, $\boldsymbol{c}$, $\boldsymbol{b}$)
end if
$\hat{\boldsymbol{b}} \leftarrow$ CNN-aided BF model($\mathbf{I}_1$, $\mathbf{I}_2$, $\boldsymbol{c}$)
for $j = 1 : \boldsymbol{w}_l$ **do**
  $\mathbf{L}$, $\mathbf{R} \leftarrow$ initialize the BP decoder using (2)
  $i \leftarrow$ index of the j-highest value in $\hat{\boldsymbol{b}}$ and mapped to the corresponding position of information bit
  $\mathbf{R}_{1,i}^{(1)} \leftarrow \infty \times (2\hat{\boldsymbol{u}}_{old,i} - 1)$
  $\hat{\boldsymbol{u}}$, $\mathbf{L}$, $\mathbf{R} \leftarrow$ BP decoder($\mathbf{L}$, $\mathbf{R}$)
  $t \leftarrow t + 1$
  if *generate training data* **then**
    $\mathbf{I}_1, \mathbf{I}_2, \boldsymbol{c} \leftarrow$ input data pre-processing($\mathbf{L}$, $\mathbf{R}$, $\mathbf{H}_{CRC}$)
    $m \leftarrow$ label Undo training data($\mathbf{R}_{1,i}^{(1)}$, $\boldsymbol{u}_i$)
    Undo data pool $\leftarrow$ ($\mathbf{I}_1$, $\mathbf{I}_2$, $\boldsymbol{c}$, m)
  end if
  if $\hat{\boldsymbol{u}}$ passes CRC **or** $t == T_{max}$ **then**
    return $\hat{\boldsymbol{u}}$
  end if
  if $l < d$ **then**
    if *Use CNN-aided Undo Model* **then**
      $\mathbf{I}_1, \mathbf{I}_2, \boldsymbol{c} \leftarrow$ input data pre-processing($\mathbf{L}$, $\mathbf{R}$, $\mathbf{H}_{CRC}$)
      $\hat{m} \leftarrow$ CNN-aided Undo model($\mathbf{I}_1$, $\mathbf{I}_2$, $\boldsymbol{c}$)
    else
      $\hat{m} \leftarrow 0$
    end if
    if $\hat{m} \leq 0.5$ **then**
      $l \leftarrow l + 1$
      $\hat{\boldsymbol{u}}_{old} \leftarrow \hat{\boldsymbol{u}}$
      Multiple-Bits BF Subtree($\mathbf{L}$, $\mathbf{R}$, $\hat{\boldsymbol{u}}_{old}$, t, l, $(\boldsymbol{u})$)
      $l \leftarrow l - 1$
    end if
  end if
end for

The input data and architecture design of the CNN-aided Undo model are as similar as the CNN-aided BF model. The only difference is that the output layer dimension is reduced from K to 1. Thus, the loss function is also revised to binary cross-entropy as below:

$$\mathcal{L}(m, \hat{m}) = -m \log \hat{m} - (1 - m)\log(1 - \hat{m}), \qquad (10)$$

where m and $\hat{m}$ denote the labeled data and predicted value for the "Undo" action, respectively.

The detailed algorithm flow for the proposed CNN-aided tree-based multiple-bits BF (CNN-Tree-MBF) mechanism is summarized in Algorithm 1 and Algorithm 2. If CNN-aided Undo model is enabled, our mechanism is abbreviated as CNN-Tree-MBF-Undo. The received signal will first go through its first round of BP decoding. After a pre-set number of iterations

TABLE I
SIMULATION PARAMETERS

Encoding (N, K, r)	Polar code (64, 32, 11)
Codeword Construction	Bhattacharya bounds algorithm @ SNR = 0 dB
Decoding Algorithm	RNN-BP [17]
Number of BP Iteration (I)	5
Training SNR (dB)	1
Testing SNR (dB)	0, 1, 2, 3
CRC Generator Polynomial	$x^{11} + x^{10} + x^9 + x^5 + 1$
Testing Codeword/SNR	240,000
Optimizer	Adam
Training and Testing Environment	DL library of Keras with i7-6700 CPU and NVIDIA GTX 2080 GPU

for BP, the CRC will be utilized to check whether the BP decoding is successful. If not, Algorithm 2 will be recursively performed in line 31 of Algorithm 2 until reaching the maximum depth d. The **for** in line 7 of Algorithm 2 means $\boldsymbol{w}_l$ bits with the highest error probability, predicted by the CNN-BF model, will be sequentially attempted in the descending order of error probability. The **if** in line 21 and line 28 will jointly determine whether to recursively perform Algorithm 2 to enter the next level and start a new subtree. The line 18 shows that this mechanism will be continuously performed until the CRC is passed or the maximum trials T_{max} in bit flipping is reached. In addition, suppose the generation of training data is required, the message $\boldsymbol{u}$ must be provided for the data labeling.

V. SIMULATION RESULTS

In this work, we utilize the recurrent neural network-based belief propagation (RNN-BP) algorithm [17] to replace the conventional BP decoding algorithms. The RNN-BP can dramatically reduce the required number of BP iterations I from 40 to 5, which can dramatically decrease the additional decoding latency caused by each flipping attempt and makes the BF mechanism more practical and efficient. Besides, the syndrome loss is also applied for the training of RNN-BP to optimize the scaling parameters from block-level, which can further enhance the decoding performance of BP algorithm [33]. The code length N and information bits K are set to 64 and 32, respectively. Besides, 11-bit CRC is added as information bits and utilized to check the decoding results. The generator polynomial is $x^{11} + x^{10} + x^9 + x^5 + 1$ as reported in [35]. Besides, the training SNR for both CNN models is set to 1 dB and the testing SNR ranges from 0 dB to 3 dB, which can be used to demonstrate that our models can generalize to unseen SNR values. The simulation setup is summarized in Table I.

A. Comparison of Prediction Accuracy Under Different Numbers of Flipping Attempts

The performance of the CNN-Tree-MBF mechanism highly relies on the prediction accuracy of CNN-BF. The higher accuracy means that our CNN-BF model has better error correction capability to dynamically identify the erroneous bits under limited flipping attempts. Thus, it also plays an important

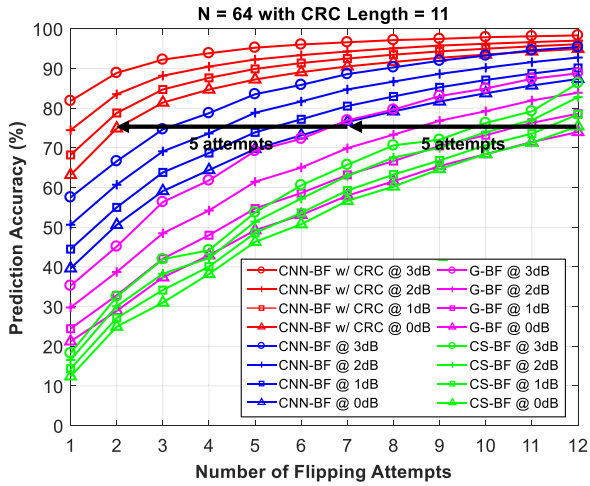

Fig. 7. Comparison of prediction accuracy between the proposed CNN-BF, opposite CS-BF [24], and G-BF [27] under different numbers of flipping attempts.

role in reducing the number of flipping attempts, namely the decoding latency. In Fig. 7, we firstly compare the prediction accuracy between the proposed CNN-BF model and state-of-the-arts, namely CS-BF [24] and generalized BF (G-BF) [27]. The number of flipping attempts is the number of tries until the decoding result passes CRC. The prediction accuracy is determined by the number of cumulative successful decodings at the number of flipping attempts as a percentage of the total samples. Note that the evaluation of prediction accuracy is based on 1-bit correctable codewords, which means the codewords can be successfully decoded by only 1 correct flipping attempt. The multiple-bits correctable codewords are not included in this experiment. The maximum number of flipping attempts T_{max} is set to 12 which is as same as |CS|. Besides, the flipping order for CS-BF [24] and G-BF [27] is based on the descending order of the error rate and the ascending order of $|\mathbf{L}_1^{(T)}|$, respectively.

In Fig. 7, the performance of the CNN-BF model without the input data of CRC results $\boldsymbol{c}$ is also compared, which is adopted in [29]. As seen in Fig. 7, all methods have better prediction accuracy as the SNR increases. However, CNN-BF predicts the correct BF position at a significantly better accuracy, especially at the earlier number of attempts, as well as having a higher ceiling for improvement. These outstanding improvements are the result of two reasons. First, the well-trained CNN model has a more accurate BF selection. Although G-BF can also dynamically select flipping bits based on $|\mathbf{L}_1^{(T)}|$, it is not accurate enough and suffers from noise at low SNR ranges. Second, CNN-BF can flip bits outside of the critical set which achieves better error correction capability over CS-BF. Both of the reasons contribute to the reduction of 5 flipping attempts for CNN-BF compared to CS-BF and G-BF. Besides, after including the CRC results $\boldsymbol{c}$ as input data, the performance improves significantly with an additional reduction of 5 flipping attempts compared to the CNN-BF model without using CRC results, which confirms our idea. Although CRC can only be used for error-detection, it is informative for CNN models to extract features and identify the error-prone bits. Besides, after transforming into an image, the information in the temporal domain can be integrated with

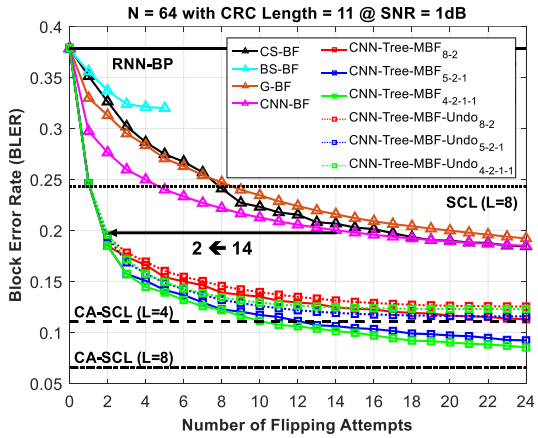

Fig. 8. Comparison of block error rate between different approaches under different numbers of flipping attempts with SNR = 1 dB.

improved prediction accuracy and thus reduce the decoding latency caused by BF.

B. Comparison of Block Error Rate Under Different Numbers of Flipping Attempts

To further quantify the above results, we realize the contribution of prediction accuracy to the block error rate. In the following experiments, the performance is evaluated on arbitrary codewords, which is not constrained to only 1-bit correctable codewords. Thus, it is a real situation and the benefits of multiple-bits BF over 1-bit BF [29] can be revealed. In this experiment, we compare the BLER under different numbers of flipping attempts with SNR set to 1 dB as shown in Fig. 8. In addition to the performance of our CNN-Tree-MBF and CNN-Tree-MBF-Undo, some baseline references and related BF-based approaches are also included for the comparison. There are four baseline references, including RNN-BP [17], SCL with list size $L = 8$, and CA-SCL with list size set to 4 and 8 [4], which are the straight lines in Fig. 8. For the related BF-based approaches, CS-BF [24], G-BF [27], 1-bit CNN-BF [29], and bit-strengthening BF (BS-BF) [23], are also compared. In this part, we adopt exhaustive CS-BF, which has better decoding performance at the cost of higher flipping attempts. Thus, the maximum number of flipping attempts is set to $2 \times |\mathrm{CS}|$. However, for BS-BF, the maximum flipping attempts is $\log N - 1$, which is corresponding to the number of stages in the BP's factor graph and is 5 in our case. For both CNN-Tree-MBF and CNN-Tree-MBF-Undo, we compare three different tree structures as indicated behind the label. For example, 5-2-1 means the tree's maximum depth d is 3 and the number of nodes in each level for $\boldsymbol{w}_1$, $\boldsymbol{w}_2$ and $\boldsymbol{w}_3$ is 5, 2, and 1, respectively.

From Fig. 8, we can observe that CA-SCL with list size set to 8 can achieve the best decoding performance. It is due to the most likely paths are kept to avoid the mistakes happening in the early stages and the decoded results can be selected by the CRC mechanism. However, compared to BP decoding algorithms, CA-SCL suffers from high latency and low throughput due to its sequential processing nature. On the other hand, all BF-based approaches based on the RNN-BP decoding algorithm achieves great improvement, but at the sacrifice of longer decoding latency. It demonstrates that the BF mechanism can provide a compromise for adjustment between decoding performance and latency.

From Fig. 8, compared to the work of 1-bit CNN-BF, our proposed CNN-Tree-MBF can significantly reduce the number of flipping attempts under the same decoding performance. Besides, it can outperform CA-SCL with list size $L = 4$. It is due to our proposed approach can benefit from the better error correction capability of multiple-bits BF and higher prediction accuracy of CNN-BF model. Besides, the tree-based Undo model also plays an important role in early terminating the useless flipping attempts. These reasons jointly contribute to the significant improvement over 1-bit CNN-BF [29].

Besides, for CNN-Tree-MBF, the BLER decreases faster as the tree's maximum depth increases due to its better error correction capability. However, the flipping error at the early level may cause more useless flipping attempts for the deeper tree structure. Thus, the number of nodes in each level decreases as the depth deepens. To best explore the tree structure, the width in each level can be dynamically adjusted by neural networks or based on the magnitude of the model prediction results, which are left as our future work.

Moreover, we can observe that the performance of CNN-Tree-MBF-Undo is slightly worse than CNN-Tree-MBF, this is due to the inaccurate prediction from CNN-aided Undo model, which degrades the error correction capability. However, it is helpful for further decreasing the flipping attempts, which will be analyzed in the following experiments.

C. Performance Under Different SNRs With Constant Maximum Number of Flipping Attempts

1) Block Error Rate: In Fig. 8, we compare the BLER under different numbers of flipping attempts to evaluate the speed of improvement for different approaches. Thus, the parameter of T_{max} can be dynamically adjusted for the tradeoff between decoding performance and decoding latency to meet different requirements. Now, we evaluate the BLER under different SNRs with T_{max} set to 24. For clarity, only one tree structure is compared for both CNN-Tree-MBF and CNN-Tree-MBF-Undo, which is set to 5-2-1. From Fig. 9, we can observe that our proposed CNN-Tree-MBF has about 1.5 dB performance gain compared with RNN-BP, which demonstrates that the BP-based decoding algorithms can be effectively improved via BF mechanisms. Compared to state-of-the-arts CS-BF, G-BF and CNN-BF, our approach also has about 0.6 dB performance gain by taking advantage of deep learning techniques and the well-designed multiple-bits BF strategy. For the BS-BF [23], though the authors demonstrate that it can approach SCL with list size $L = 16$ under code length $N = 2048$, it only has a slight improvement in our experiments due to the limited number of flipping attempts. However, our approach can further outperform CA-SCL with $L = 4$, which has been seen as state-of-the-art for the polar decoder. Later, we will show that as T_{max} increases, our approach can even outperform CA-SCL with $L = 8$.

2) Average Flipping Attempts: To evaluate the impact of additional decoding latency caused by BF, we examine the

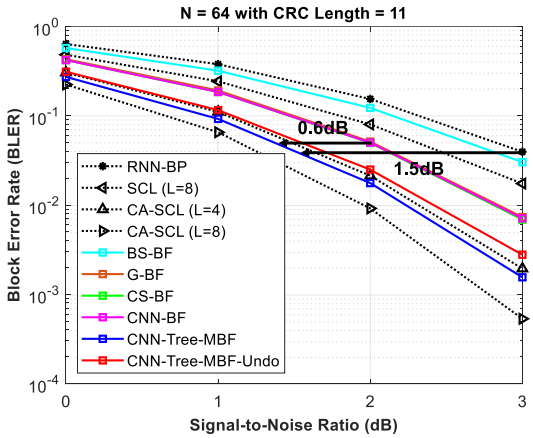

Fig. 9. Comparison of block error rate under different SNRs with $T_{max} = 24$.

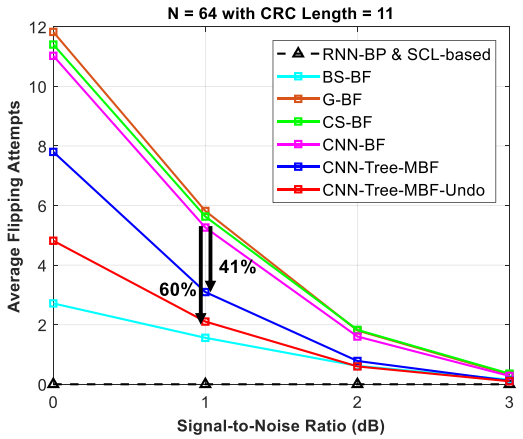

Fig. 10. Comparison of average flipping attempts under different SNRs with $T_{max} = 24$.

average flipping attempts T_{avg} for each approach in Fig. 9. From Fig. 10, the average flipping attempts decreases rapidly as SNR increases. For example, at SNR = 3 dB, the flipping attempts for CNN-BF, CNN-Tree-MBF and CNN-Tree-MBF-Undo are merely 0.28, 0.11 and 0.10, respectively. It shows that the average increase in decoding latency is small enough. However, it still contributes to significant improvement in decoding performance as shown in Fig. 9. The analysis of overall decoding latency, including the inference of CNN model, will be provided in Section V.F. From Fig. 10, the average flipping attempts of CS-BF, G-BF and CNN-BF are very close, this can be indicated in Fig. 8. Because the average flipping attempts can be calculated by integrating the BLER over the number of flipping attempts. As the number of flipping attempts increases, the BLER decreases slowly and thus the uncorrectable codewords will dominate the average flipping attempts, which make them have similar performance. Suppose T_{max} is set smaller, the gap between them will become more apparent. On the other hand, compared to CNN-BF, our CNN-Tree-MBF can significantly reduce the average flipping attempts by 41% due to the rapidly decreased BLER and the better error correction capability as shown in Fig. 8. Moreover, our CNN-Tree-MBF-Undo can utilize CNN-aided Undo model for early termination to further

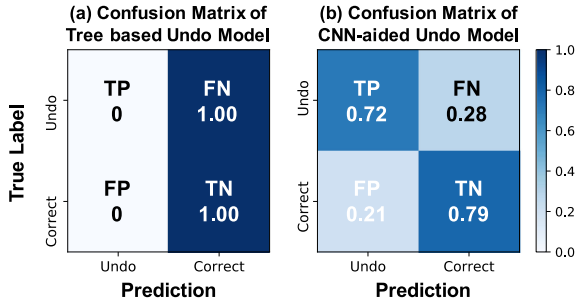

Fig. 11. Comparison of confusion matrix between (a) tree-based Undo model, and (b) tree-based CNN-aided Undo model with SNR = 1 dB.

reduce 19% average flipping attempts at the cost of slightly degraded BLER. For the BS-BF, it has the lowest average flipping attempts due to T_{max} is only 5.

D. Analysis Between Tree-Based Undo Model and CNN-Aided Undo Model

The proposed tree-based Undo model is a criteria-based model that performs the "Undo" action only when the flipping attempt reaches the maximum depth; otherwise, it successively flips another bit. On the other hand, before reaching the maximum depth, if the CNN-aided Undo model is applied, it can dynamically determine the action by examing the metadata $\mathbf{L}$, $\mathbf{R}$ and CRC results $\boldsymbol{c}$ from the last BP decoding. Thus, it can effectively avoid the useless flipping attempts but slightly degrade the error correction capability. For a better evaluation of the pros and cons of these two strategies, the confusion matrices of them are shown in Fig. 11, which is greatly helpful for the explanation of our previous experiments.

In this experiment, the performance is evaluated based on the codewords that have not reached the maximum depth. Thus, the last flipped bit is always considered as the correct position and thus the "No Undo" action is always performed by the tree-based Undo model as shown in Fig. 11. The meaning behind TP, TN, FP, and FN can refer to Section IV.B. The higher TP and TN can reduce the number of useless flipping attempts and avoid degrading the error correction capability, respectively. Vice versa, the higher FN and FP will result in useless flipping attempts and degrade the error correction capability, respectively. From Fig. 11, because the tree-based Undo model always performs "No Undo" action, it sacrifices TP to obtain 100% TN. On the other hand, the CNN-aided Undo model strikes a balance between TP and TN, which improves TP to avoid useless flipping attempts at the cost of increased FP. Thus, it's confusion matrix can clearly illustrate the simulation results shown in Fig. 9 and Fig. 10. Meanwhile, it also demonstrates that there is still room for future extension to further enhance the prediction accuracy of the CNN-aided Undo model.

E. Performance Under Different SNRs With Various Maximum Number of Flipping Attempts

1) Block Error Rate: We repeat the experiments in Fig. 9 and Fig. 10 but at the various maximum number of flipping attempts T_{max} to demonstrate its great flexibility to dynamically adjust

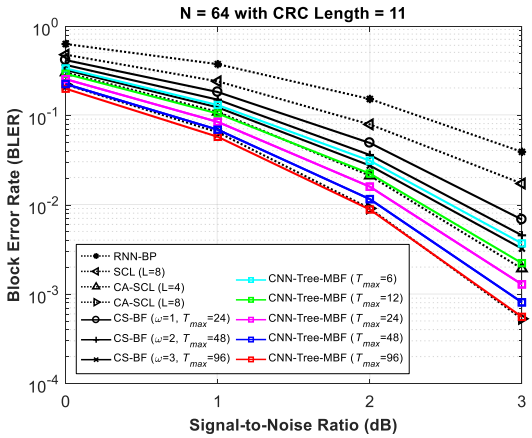

Fig. 12. Comparison of block error rate under different SNRs with the various maximum number of flipping attempts.

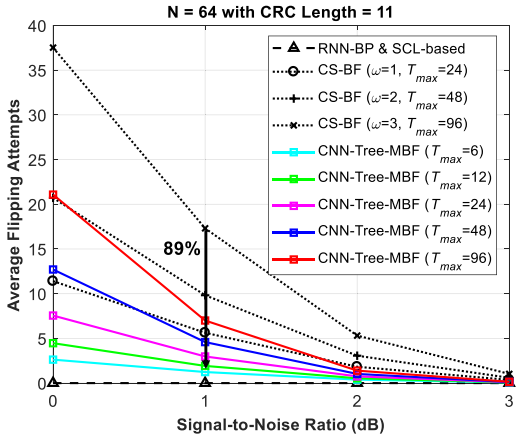

Fig. 13. Comparison of average flipping attempts under different SNRs with the various maximum number of flipping attempts.

between decoding performance and decoding latency. In Fig. 12, the performance of RNN-BP, SCL, and CA-SCL is also derived as the baseline references. However, for clarity, G-BF, BS-BF and CNN-BF are removed. On the other hand, the performance of CS-BF under different number of flipped bits ω, corresponding to $2^{\omega} \times |\text{CS}|$ maximum number of flipping attempts, is also derived for a fair comparison. Due to the maximum number of flipping attempts increases, we increase the tree's maximum depth to 4 to better exploit the benefit of multiple-bits BF with the structure set to 14-2-1-1. From Fig. 12, we can observe that the proposed CNN-Tree-MBF has similar performance as CA-SCL with $L = 4$ and 8, which improves about 1.4 dB and 1.8 dB compared to RNN-BP when $T_{max} = 12$ and 96, respectively. Meanwhile, when $T_{max} = 12$, it can outperform CS-BF with $\omega = 3$ which is equivalent to $T_{max} = 96$. Thus, our approach can significantly improve the error correction capability with less flipping attempts. Besides, it also provides a wide range for the adjustment of the desired BLER by setting the different number of T_{max}.

2) Average Flipping Attempts: Then, the average flipping attempts for different approaches are shown in Fig. 13. We can observe that the additional decoding latency for our proposed CNN-Tree-MBF is still small enough even for $T_{max} = 96$. However, it can still contribute to significant improvement as shown in Fig. 12. Besides, our proposed CNN-Tree-MBF with $T_{max} = 12$ can outperform CS-BF with $\omega = 3$ at about 89% reduction in average flipping attempts.

F. Analysis of Computational Complexity, Memory Overhead, and Decoding Latency

Finally, we further analyze the computational complexity, memory overhead, and decoding latency between our proposed CNN-Tree-MBF and the related works, including RNN-BP [17], CS-BF [24], and CA-SCL [4], which are summarized in Table II. To achieve comparable performance as CA-SCL with $L = 8$, we set $T_{max} = 96$ for analysis. Note that the compared CA-SCL [4] is a general case and many hardware optimization techniques can be found in [36]–[37] to increase throughput and reduce decoding latency.

1) Computational Complexity: We use the number of floating-point operations (FLOPs) to represent computational complexity. The FLOPs for each RNN-BP decoding are $2INn$ and the overall FLOPs for CS-BF increase linearly with T_{avg}. For our proposed CNN-Tree-MBF, it has additional overhead for the inference of CNN model, which increases linearly with the average times of CNN inference T_{CNN} and thus is based on the tree structure as shown in Fig. 6(a). Compared to other CNN models used in computer vision with multiple GFLOPs, our model is tiny with only 9.8M FLOPs. However, it is still greater than other conventional approaches.

2) Memory Overhead: We use the number of parameters to evaluate memory overhead. RNN-BP stores **L, R** messages for the calculation of updated messages, which is about $N(n+1)$ because the memory can be shared between **L, R** messages [38]. CS-BF requires additional memory for the storage of the flipping table, which is around ωT_{max}. Our proposed CNN model requires the storage of both **L, R** messages in each iteration as the input data, which is $2I$ times than RNN-BP. Besides, it also requires additional memory for the prediction results, the decoded bit value for the determination of flipping direction, and the storage of CNN parameters, which is T_{max}, K and around 0.3M, respectively. For the case of CA-SCL, we refer to [24], which is $[N + (N-1)L] + L + (2N-1)L$.

3) Decoding Latency: We evaluate the decoding latency in terms of the required number of time steps and CPU computing time. For the iterative RNN-BP decoding process, it requires $2In$ time steps and thus CS-BF is $2InT_{avg}$. Our CNN model requires additional time steps for the inference which is dependent on the number of NN layers [39]–[40] and is 10 in our case. Besides, it also increases linearly with T_{CNN}. For a general CA-SCL, it requires $2N-2$ time steps. The CPU computing time is also listed in Table II. Note that the sorting time of the output probabilities is omitted because it is far smaller than the model inference and BP decoding time. From Table II, it shows that our approach can achieve comparable performance as CA-SCL with fewer time steps and shorter computing time at high SNR.

In summary, although our CNN model consumes more hardware resources, many techniques have been proposed, such as pruning and quantization, which can successfully compress the neural networks by more than 40 times without performance

TABLE II
ANALYSIS OF COMPUTATIONAL COMPLEXITY, MEMORY OVERHEAD, AND DECODING LATENCY

	Floating-Point Operations (FLOPs)		Memory Overhead	Decoding Latency (Time Steps)		CPU Computing Time (s)	
SNR	0dB	3dB		0dB	3dB	0dB	3dB
RNN-BP [17]	$2INn = 3840$		$N(n+1) = 448$	$2In = 60$		0.0015	
CS-BF [24]	$2INnT_{avg}$		$N(n+1) + \omega T_{max} = 736$	$2InT_{avg}$		$0.0015T_{avg}$	
	144080	3950		2251.2	61.8	0.0563	0.0015
Proposed CNN-Tree-MBF	$2INnT_{avg} + 9.8\text{M} \times T_{CNN}$		$2IN(n+1) + T_{max} + K$ $+ 0.3\text{M} \approx 0.3\text{M}$	$2InT_{avg} + 10T_{CNN}$		$0.0015T_{avg} + 0.0013T_{CNN}$	
	170.4M	1.3M		2424.9	63.1	0.0789	0.0017
CA-SCL [4]	$L\left(Nn + \frac{3N}{2} + \frac{5K}{2}\right) - \frac{1}{2}K = 4464$		$N + 3NL - L = 1592$	$2N + K - 2 = 158$		0.0283	

$*N = 64$, $K = 32$, $n = 6$, $I = 5$, $L = 8$, $\omega = 3$, $T_{max} = 96$

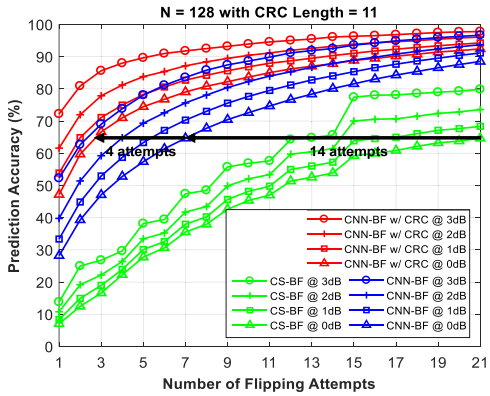

Fig. 14. Comparison of prediction accuracy between the proposed CNN-BF and the opposite CS-BF [24] under different numbers of flipping attempts.

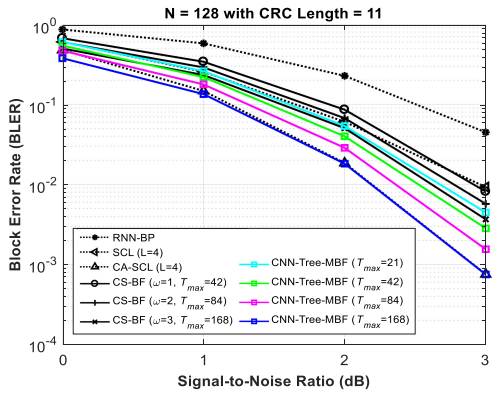

Fig. 16. Comparison of block error rate under different SNRs with the various maximum number of flipping attempts.

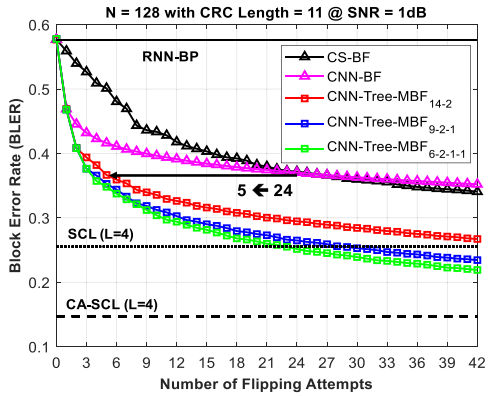

Fig. 15. Comparison of block error rate between different approaches under different numbers of flipping attempts with SNR = 1 dB.

degradation. These techniques can be applied to our CNN model for the reduction of hardware complexity. For the future extension, we suggest that the CNN model can be replaced by some lightweight ML models, such as tree-based models, for the tradeoff between complexity and prediction accuracy.

G. Evaluation of Scalability

To demonstrate the scalability of our proposed approach, we also repeat the simulations by increasing code length from 64 to 128 in Fig. 14 to Fig. 16. We can observe that the simulation results under $N = 128$ have a similar trend as $N = 64$. Besides, the gap between our proposed CNN-Tree-MBF and CS-BF becomes larger, which demonstrates the great advantage of neural networks under more difficult conditions.

VI. CONCLUSION

In this paper, we present a novel convolutional neural network-aided tree-based bit-flipping decoder, which can effectively exploit the benefit of multiple-bits BF. With carefully designed input data and domain-specific data pre-processing, our model can learn from the BP metadata and the CRC results to correctly predict flipping position, with more accuracy than the prior critical set method and our previous CNN-BF. Besides, the tree structure for flipping strategy can strike a good balance between the benefit of multiple-bits BF and the increased flipping attempts caused by wrongly flipped bits. Therefore, it can achieve more than 1.5 dB gain over RNN-BP algorithm with slightly increased decoding latency.

REFERENCES

[1] E. Arıkan, "Channel polarization: A method for constructing capacity-achieving codes for symmetric binary-input memoryless channels," *IEEE Trans. Inf. Theory*, vol. 55, no. 7, pp. 3051–3073, Jul. 2009.
[2] "Final report of 3GPP TSG RAN WG1 #87 v1.0.0," Reno, USA, Nov. 2016. [Online]. Available: https://www.3gpp.org/ftp/tsg_ran/WG1_RL1/TSGR1_87/Report/Final_Minutes_report_RAN1%2387_v100.zip
[3] K. Niu and K. Chen, "CRC-assisted decoding of polar codes," *IEEE Commun. Lett.*, vol. 16, no. 10, pp. 1668–1671, Oct. 2012.
[4] I. Tal and A. Vardy, "List decoding of polar codes," *IEEE Trans. Inf. Theory*, vol. 61, no. 5, pp. 2213–2226, May 2015.

[5] O. Afisiadis, A. Balatsoukas-Stimming, and A. Burg, "A low-complexity improved successive cancellation decoder for polar codes," in *Proc. 48th Asilomar Conf. Signals, Syst. Comput.*, Nov. 2014, pp. 2116–2120.
[6] L. Chandesris, V. Savin, and D. Declercq, "An improved SCFlip decoder for polar codes," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2016, pp. 1–6.
[7] Z. Zhang, K. Qin, L. Zhang, H. Zhang, and G. T. Chen, "Progressive bit-flipping decoding of polar codes over layered critical sets," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2017, pp. 1–6.
[8] Z. Zhang, K. Qin, L. Zhang, and G. T. Chen, "Progressive bit-flipping decoding of polar codes: A critical-set based tree search approach," *IEEE Access*, vol. 6, pp. 57738–57750, 2018.
[9] L. Chandesris, V. Savin, and D. Declercq, "Dynamic-SCFlip decoding of polar codes," *IEEE Trans. Commun.*, vol. 66, no. 6, pp. 2333–2345, Jun. 2018.
[10] F. Ercan, C. Condo, and W. J. Gross, "Improved bit-flipping algorithm for successive cancellation decoding of polar codes," *IEEE Trans. Commun.*, vol. 67, no. 1, pp. 61–72, Jan. 2019.
[11] C.-H. Chen, C.-F. Teng, and A.-Y. A. Wu, "Low-complexity LSTM-assisted bit-flipping algorithm for successive cancellation list polar decoder," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, May 2020, pp. 1708–1712.
[12] X. Wang *et al.*, "Learning to flip successive cancellation decoding of polar codes with LSTM networks," in *Proc. IEEE 30th Annu. Int. Symp. Pers., Indoor Mobile Radio Commun.*, Istanbul, Turkey, 2019, pp. 1–5.
[13] B. He, S. Wu, Y. Deng, H. Yin, J. Jiao, and Q. Zhang, "A machine learning based multi-flips successive cancellation decoding scheme of polar codes," in *Proc. IEEE 91st Veh. Technol. Conf.*, May 2020, pp. 1–5.
[14] E. Arıkan, "Polar codes: A pipelined implementation," in *Proc. 4th Int. Symp. Broad. Commun.*, Jul. 2010, pp. 413–416.
[15] W. Xu, Z. Wu, Y.-L. Ueng, X. You, AND, and C. Zhang, "Improved polar decoder based on deep learning," in *Proc. IEEE Int. Workshop Signal Process. Syst.*, Oct. 2017, pp. 1–6.
[16] E. Nachmani, E. Marciano, L. Lugosch, W. Gross, D. Burshtein, and Y. Be'ery, "Deep learning methods for improved decoding of linear codes," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 119–131, Feb. 2018.
[17] C.-F. Teng, C.-H. D. Wu, A. K.-S. Ho, and A.-Y. A. Wu, "Low-complexity recurrent neural network-based polar decoder with weight quantization mechanism," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, May 2019, pp. 1413–1417.
[18] N. Doan, S. A. Hashemi, E. N. Mambou, T. Tonnellier, and W. J. Gross, "Neural belief propagation decoding of CRC-polar concatenated codes," in *Proc. IEEE Int. Conf. Commun.*, Shanghai, China, 2019, pp. 1–6.
[19] A. Elkelesh, M. Ebada, S. Cammerer, and S. T. Brink, "Belief propagation list decoding of polar codes," *IEEE Commun. Lett.*, vol. 22, no. 8, pp. 1536–1539, Aug. 2018.
[20] N. Doan, S. A. Hashemi, M. Mondelli, and W. J. Gross, "On the decoding of polar codes on permuted factor graphs," in *Proc. IEEE Global Commun. Conf.*, Abu Dhabi, United Arab Emirates, 2018, pp. 1–6.
[21] Y. Ren, Y. Shen, Z. Zhang, X. You, and C. Zhang, "Efficient belief propagation polar decoder with loop simplification based factor graphs," *IEEE Trans. Veh. Technol.*, vol. 69, no. 5, pp. 5657–5660, 2020.
[22] S. Sun, S.-G. Cho, and Z. Zhang, "Post-processing methods for improving coding gain in belief propagation decoding of polar codes," in *Proc. IEEE Global Commun. Conf.*, Dec. 2017, pp. 1–6.
[23] X. Wang, Z. Zheng, J. Li, L. Shan, and Z. Li, "Belief propagation bit-strengthening decoder for polar codes," *IEEE Commun. Lett.*, vol. 23, no. 11, pp. 1958–1961, Nov. 2019.
[24] Y. Yu, Z. Pan, N. Liu, and X. You, "Belief propagation bit-flip decoder for polar codes," *IEEE Access*, vol. 7, pp. 10937–10946, 2019.
[25] J. Zhang and M. Wang, "Belief propagation decoder with multiple bit-flipping sets and stopping criteria for polar codes," *IEEE Access*, vol. 8, pp. 83710–83717, 2020.
[26] D. Zheng, K. Qin, and Z. Zhang, "BP list decoding of polar codes with adaptive bit splitting over critical set," in *Proc. IEEE 11th Int. Conf. Wireless Commun. Signal Process.*, Oct. 2019, pp. 1–7.
[27] Y. Shen, W. Song, Y. Ren, H. Ji, X. You, and C. Zhang, "Enhanced belief propagation decoder for 5G polar codes with bit-flipping," *IEEE Trans. Circuits Syst., II, Exp. Briefs*, vol. 67, no. 5, pp. 901–905, May 2020.
[28] Y. Shen *et al.*, "Improved belief propagation polar decoders with bit-flipping algorithms," *IEEE Trans. Commun.*, vol. 68, no. 11, pp. 6699–6713, Nov. 2020.
[29] C.-F. Teng, K.-S. Ho, C.-H. Wu, S.-S. Wong, and A.-Y. Wu, "Convolutional neural network-aided bit-flipping for belief propagation decoding of polar codes," *arXiv: 1911.01704,* 2019. [Online]. Available: https://arxiv.org/abs/1911.01704
[30] J. Chen, Z. Wei, S. Li, and B. Cao, "Artificial intelligence aided joint bit rate selection and radio resource allocation for adaptive video streaming over F-RANs," *IEEE Wireless Commun*, vol. 27, no. 2, pp. 36–43, Apr. 2020.
[31] L. Huang, H. Zhang, R. Li, Y. Ge, and J. Wang, "AI coding: Learning to construct error correction codes," *IEEE Trans. Commun.*, vol. 68, no. 1, pp. 26–39, Jan. 2020.
[32] C. Zhang, Y. Ueng, C. Studer, and A. Burg, "Artificial intelligence for 5G and beyond 5G: Implementations, algorithms, and optimizations," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 10, no. 2, pp. 149–163, Jun. 2020.
[33] M. Bojarski *et al.*, "End to end learning for self-driving cars," *arXiv: 1604.07316*, 2016. [Online]. Available: https://arxiv.org/abs/1604.07316
[34] C.-F. Teng and Y.-L. Chen, "Syndrome-enabled unsupervised learning for neural network-based polar decoder and jointly optimized blind equalizer," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 10, no. 2, pp. 177–188, Jun. 2020.
[35] V. Bioglio, C. Condo, and I. Land, "Design of polar codes in 5G new radio," *IEEE Commun. Surveys Tuts*, 2020.
[36] S. A. Hashemi, C. Condo, and W. J. Gross, "Fast and flexible successive-cancellation list decoders for polar codes," *IEEE Trans. Signal Process.*, vol. 65, no. 21, pp. 5756–5769, Nov. 2017.
[37] S. A. Hashemi, C. Condo, M. Mondelli, and W. J. Gross, "Rate-flexible fast polar decoders," *IEEE Trans. Signal Process.*, vol. 67, no. 22, pp. 5689–5701, Nov. 2019.
[38] C.-F. Teng, C.-H. Chen, and A.-Y. A. Wu, "An ultra-low latency 7.8-13.6 pJ/b reconfigurable neural network-assisted polar decoder with multi-code length support," in *Proc. IEEE Symp. VLSI Circuits*, 2020, pp. 1–2.
[39] S. Cammerer, T. Gruber, J. Hoydis, and S. T. Brink, "Scaling deep learning-based decoding of polar codes via partitioning," in *Proc. IEEE Global Commun. Conf.*, Dec. 2017, pp. 1–6.
[40] N. Doan, S. A. Hashemi, and W. J. Gross, "Neural successive cancellation decoding of polar codes," in *Proc. IEEE Int. Workshop Signal Process. Adv. Wireless Commun.*, 2018, pp. 1–5.

Chieh-Fang Teng (Student Member, IEEE) received the B.S. degree in electrical engineering from National Taiwan University, Taipei, Taiwan, in 2017. He is currently working toward the Ph.D. degree with the Graduate Institute of Electronics Engineering, National Taiwan University. His research interests include the areas of Internet of Things, VLSI architecture for DSP, and machine learning assisted wireless communication systems design.

An-Yeu (Andy) Wu (Fellow, IEEE) received the B.S. degree from National Taiwan University in 1987, and the M.S. and Ph.D. degrees from the University of Maryland, College Park in 1992 and 1995, respectively, all in electrical engineering. In August 2000, he joined the faculty of the Department of Electrical Engineering and the Graduate Institute of Electronics Engineering, National Taiwan University, where he is currently a Distinguished Professor. His research interests include VLSI architectures for signal processing and communications, and adaptive/multirate signal processing. He has authored or coauthored more than 190 refereed journal and conference papers in above research areas, together with five book chapters and 16 granted US patents. From August 2007 to December 2009, he was on leave from NTU and served as the Deputy General Director of SoC Technology Center (STC), Industrial Technology Research Institute (ITRI), Hsinchu, Taiwan. In 2010, he was the recipient of the Outstanding EE Professor Award from The Chinese Institute of Electrical Engineering (CIEE), Taiwan. From 2012 to 2014, he served as the Chair of VLSI Systems and Applications (VSA) Technical Committee (TC), one of the largest TCs in IEEE Circuits and Systems (CAS) Society. In 2015, he was elevated to IEEE Fellow for his contributions to DSP algorithms and VLSI designs for communication IC/SoC. From 2016 to 2019, he served as Director of Graduate Institute of Electronics Engineering (GIEE), National Taiwan University. He is currently the Editor-in-Chief (EiC) of IEEE JOURNAL ON EMERGING AND SELECTED TOPICS IN CIRCUITS AND SYSTEMS (JETCAS).